

JessiePage

User Manual

Text-first interactive geometry and computer algebra



Web application: mrsmkl.github.io/jessiepage

Manual generated from the JessiePage feature set.

Contents

1 What JessiePage is	2
1.1 The workspace	2
2 First construction	2
2.1 Changing appearance	2
3 Working with the canvas	3
3.1 Dragging points or text and source readback	3
4 Pages, saving, and links	3
5 CAS basics	4
5.1 Core operations	4
5.2 Equations and solutions	4
6 Graphing CAS results	5
6.1 Sampled points	5
7 Plain text and LaTeX-style input	5
8 Finite sums and products	5
9 Matrices, complex numbers, and assumptions	6
9.1 Matrices	6
9.2 Complex numbers	6
9.3 Assumptions	6
10 Sharing variables with JessieCode	7
10.1 Geometry passed to the CAS	7
11 Euclid Book I	7
12 Errors and troubleshooting	8
12.1 A line shows a CAS error	8
12.2 A Jessie construction does not redraw	8
12.3 A result has no Graph switch	8
12.4 Typing or navigation feels slow	8
12.5 A saved page disappeared	8
13 Compact reference	8
13.1 Common JessieCode constructors	9
13.2 Keyboard and editing reminders	9
14 Further information	9

1 What JessiePage is

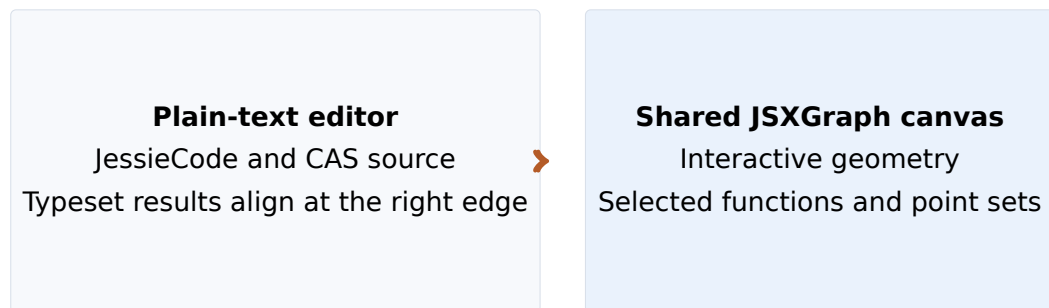
JessiePage combines two mathematical tools in one continuous text document:

- **JessieCode with JSXGraph** constructs interactive points, lines, circles, polygons, curves, sliders, and other geometry.
- **CortexJS Compute Engine** evaluates algebra, equations, calculus, matrices, collections, and finite sums or products.

The left side is always ordinary editable text. Results are typeset beside the source line that produced them. Graphable CAS results have a small **Graph** switch; enabled results share the large canvas on the right.

There are no notebook cells and no formula editor. A line is interpreted automatically from its syntax.

1.1 The workspace



Pages, status, source readback, and local autosave Pan, zoom, reset, and fullscreen Graph

2 First construction

Open JessiePage and create or select a page. Enter the following program:

```
A = point(-3, -1);
B = point(3, -1);
C = point(0, 3);
triangle = polygon(A, B, C);
M = midpoint(A, B);
c = circle(M, C);
```

JessieCode statements normally end with a semicolon. Named objects can be reused by later statements. Editing a valid line rebuilds the construction automatically.

2.1 Changing appearance

JessieCode attributes follow an object in a double-angle block:

```
AB = segment(A, B) <<strokeWidth: 3, strokeColor: '#245ea6'>>;
c = circle(M, C) <<dash: 2, strokeColor: '#b45b27'>>;
triangle = polygon(A, B, C)
  <<fillColor: '#dbeafe', fillOpacity: 0.35>>;
```

Use the built-in example pages as working syntax references. They are ordinary editable programs rather than locked demonstrations.

3 Working with the canvas

- Drag an empty area of the canvas to pan.
- Use the mouse wheel or a two-finger pinch to zoom.
- Use **+** and **-** on the canvas for stepped zoom.
- Use **100%** to restore the page's saved view.
- Use the canvas fullscreen control to focus on the construction.
- Drag the divider between editor and canvas to resize the two areas.

3.1 Dragging points or text and source readback

A point written with two literal numeric coordinates is source-backed:

```
A = point(-2, 1);
```

Dragging this point updates those coordinates in the text. The **Points** area also lists the current coordinates of named points and gliders.

Constructed points are intentionally not rewritten. For example, the source of a midpoint or a point defined by `map () -> ...` remains the construction rule, not its temporary position.

A standalone text widget with two literal coordinates works the same way:

```
title = text(-4, 5, 'Move me');  
text(-4, -5, map () -> 'Length = ' + dist(A, B));
```

Dragging either widget updates only its first two arguments. The displayed text may be static or dynamic. A computed position such as `text(map () -> X(A), ...)` remains controlled by its expression and is not rewritten.

Stable constructions

Use ordinary points and literal-position text for inputs that students should drag. Use midpoints, intersections, gliders, map-defined points, and computed text positions for derived objects. This keeps the source readable and preserves the construction logic.

4 Pages, saving, and links

The page selector contains user pages, general examples, Euclid postulates, common notions, and Book I propositions.

- Press the top-bar **+** to create a new page.
- Edit the page name directly in the top bar.

- Source, page names, selected CAS graphs, and canvas views are saved automatically in browser local storage.
- Press **Link** to open a shareable URL containing the current program and graph selections.

Local storage is not cloud storage

Saved pages remain in the current browser profile. Clearing site data removes them. A shared link carries one program in its URL, but does not upload the rest of the page collection.

5 CAS basics

CAS expressions do not need semicolons. Definitions share one scope from top to bottom:

```
f = x^3 + 2*x^2 - x + 4
g = differentiate(f, x)
h = differentiate(g, x)
simplify(h)
```

The rendered result for each executable CAS line appears at its right. A thin connector follows the end of the source toward the result.

5.1 Core operations

Operation	Meaning
<code>simplify(expr)</code>	Apply general simplification.
<code>expand(expr)</code>	Expand products and powers.
<code>factor(expr, x)</code>	Factor an expression, optionally with respect to a variable.
<code>together(expr)</code>	Combine rational terms over a common denominator.
<code>evaluate(expr)</code>	Evaluate exactly where possible.
<code>numeric(expr, digits)</code>	Request a numerical approximation and optional precision.
<code>substitute(expr, x, value)</code>	Replace a variable and evaluate the result.
<code>solve(expr, x)</code>	Solve an expression equal to zero, or solve an equation.
<code>differentiate(expr, x)</code>	Differentiate with respect to a variable.
<code>integrate(expr, x)</code>	Compute an indefinite integral.
<code>integrate(expr, x, a, b)</code>	Compute a definite integral.
<code>limit(expr, x, value)</code>	Compute a limit.

5.2 Equations and solutions

`solve()` accepts either an equation or an expression interpreted as equal to zero:

```
f = (x - 3)*(x + 2)*(x - 1)
solve(f, x)

solve(x^2 = 9, x)
```

All representable solutions are returned as a set. Real solutions can be graphed together as points on the horizontal axis.

6 Graphing CAS results

If a CAS result is a function of one variable, a coordinate pair, a complex number, or a set of coordinate pairs, JessiePage offers a **Graph** switch beside it.

```
f = cos(x)
g = e^x
f + g
f*g
points = {(-3,2), (-1,-2), (1,3), (3,1)}
```

Enable several switches to compare all selected results on the same axes. Each function is compiled once before JSXGraph samples it, which keeps pan and zoom responsive.

Use `plot(f)` only when graph activation should be part of persisted or shared source:

```
f = x^2 - 4
plot(f)
```

6.1 Sampled points

The collection helpers can create a complete point set from one function definition:

```
f = x^2 - 4
xs = range(-3, 3)
ys = map(f, xs)
samples = zip(xs, ys)
```

Enable the graph for `samples` to draw all pairs together.

7 Plain text and LaTeX-style input

Common function notation works as ordinary text:

```
f = sin(x) + cos(x)
g = e^x
sqrt(2)
```

LaTeX-style expressions can be mixed into the same source:

```
f = \frac{x^2}{2}
g = \sin(x)
f + g
\cos(x)
```

The source remains plain text even when the displayed result is typeset. This makes copying, editing, and versioning predictable.

8 Finite sums and products

JessiePage supplies plain-text wrappers because CortexJS's native function-call parser does not interpret the words `sum` and `product` as these operations by itself:

```
term = x^k
s = sum(term, k, 1, 4)
sum(k^2, k, 1, 10)

factor = x + k
p = product(factor, k, 1, 3)
product(k, k, 1, 6)
```

The equivalent LaTeX notation is also accepted directly:

```
\sum_{k=1}^{10} k
\prod_{k=1}^{6} k
```

If a finite result still contains exactly one free variable, it receives a graph switch. For example, `sum(x^k, k, 1, 4)` becomes a polynomial in x .

9 Matrices, complex numbers, and assumptions

9.1 Matrices

Matrices use nested lists:

```
A = [[1,2],[3,4]]
determinant(A)
inverse(A)
transpose(A)

B = [[2,1],[1,2]]
eigenvalues(B)
```

Matrix results are typeset as matrices and are not mistaken for sets of points.

9.2 Complex numbers

Use i for the imaginary unit:

```
z = 3 + 2*i
w = (1-i)*z
z
w
```

A non-real complex result can be graphed as (real, imaginary) on the shared canvas.

9.3 Assumptions

Assumptions affect subsequent CAS lines until forgotten:

```
assume(x > 0)
simplify(\sqrt{x^2})
forget(x)
simplify(\sqrt{x^2})
```

10 Sharing variables with JessieCode

Compatible constant values are defined in both systems. This lets one number control algebra and geometry:

```
scale = 2
offset = 1
height = scale + offset

A = point(-scale, offset);
B = point(scale, height);
segment(A, B);

f = x^2 + height
```

Numbers, booleans, and numeric collections cross from CAS into JessieCode when they have a safe representation. Multi-letter names such as `scale` and `offset` are substituted as whole identifiers rather than being split into products of letters.

10.1 Geometry passed to the CAS

Straightforward Jessie geometry receives a mathematical CAS representation:

Jessie object	CAS representation
Point	Coordinate pair.
Line	Implicit line equation in x and y .
Segment	Equation of the infinite carrier line; endpoint bounds are not included.
Circle	Implicit circle equation.
Quoted function graph	Equation of the form $y = f(x)$.

This permits later symbolic operations:

```
A = point(-2, 0);
B = point(2, 2);
carrier = line(A, B);
solve(carrier, y)

c = circle(A, 3);
solve(c, y)
```

When a simple source-backed point moves, its dependent CAS equations refresh after the point is released.

Deliberate limits

Polygons, gliders, sliders, arbitrary dynamic maps, Jessie object aliases, and object collections are not forced into CAS expressions. They stay in JessieCode because a single algebraic value would be incomplete or misleading.

11 Euclid Book I

The built-in Euclid collection contains:

- the five postulates;
- the five common notions; and
- all 48 propositions of Book I.

Each page is an editable JessieCode construction. Movable inputs are ordinary points wherever practical, so dragging them can update their coordinates in the source. Captions and numerical statements use dynamic text when the geometry changes.

The diagrams are intended as interactive illustrations of the constructions and claims. The postulates and common notions are assumptions, and the diagrams are not substitutes for Euclid's proofs.

12 Errors and troubleshooting

12.1 A line shows a CAS error

Check parentheses, braces, argument count, and variable spelling. CAS errors are attached to their source line and do not prevent later independent CAS lines from being analyzed.

12.2 A Jessie construction does not redraw

JessieCode statements generally require semicolons. Check the highlighted error line, then compare the syntax with a built-in geometry example.

12.3 A result has no Graph switch

The value may be scalar, a matrix, a relation, a function of more than one free variable, or another result without an unambiguous two-dimensional graph. Use substitution or solve for one variable first.

12.4 Typing or navigation feels slow

Keep functions in shared definitions instead of repeating large expressions. Graph callbacks are compiled once, but extremely large symbolic results can still be expensive. Turn off graphs that are not currently needed.

12.5 A saved page disappeared

Pages are stored in the browser profile. Check whether site data was cleared, private browsing was used, or a different browser/profile is open. Use **Link** for individual programs that need a portable copy.

13 Compact reference

13.1 Common JessieCode constructors

Constructor	Purpose
<code>point(x,y)</code>	Free or computed point.
<code>line(A,B)</code>	Infinite line through two points.
<code>segment(A,B)</code>	Segment between two points.
<code>circle(O,A)</code>	Circle with center O through A.
<code>polygon(A,B,C,...)</code>	Polygon with the listed vertices.
<code>midpoint(A,B)</code>	Midpoint of two points.
<code>intersection(a,b,n)</code>	Indexed intersection of two objects.
<code>perpendicular(l,A)</code>	Perpendicular to a line through a point.
<code>glider(x,y,obj)</code>	Point constrained to an object.
<code>tangent(P)</code>	Tangent determined by a glider or contact point.
<code>slider(p1,p2,[min,value,max])</code>	Interactive numeric control.
<code>functiongraph('expr')</code>	Graph of a quoted function.
<code>curve(fx,fy,t0,t1)</code>	Parametric curve.
<code>text(x,y,value)</code>	Static or dynamic text.
<code>angle(A,B,C)</code>	Angle with vertex B.

13.2 Keyboard and editing reminders

- Source is plain text and accepts ordinary copy and paste.
- Autocomplete suggests constructors, CAS calls, functions, and names already defined in the source.
- Pasting suppresses autocomplete so it cannot interfere with immediate evaluation.
- Results are overlays; selecting and editing the underlying source does not require entering a separate result cell.

14 Further information

- JessiePage: <https://mrsmkl.github.io/jessiepage/>
- Source repository: <https://github.com/mrsmkl/jessiepage>
- JessieCode reference: <https://jsxgraph.org/docs/symbols/JXG.JessieCode.html>
- CortexJS Compute Engine: <https://cortexjs.io/compute-engine/>